



Министерство науки и высшего образования Российской Федерации
Федеральное автономное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 9
по курсу «Методы оптимизации»
«Метод Ньютона»

Студент группы ИУ9-81Б Кириленко У.О.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Метод Ньютона. Сравнить с модификацией, когда обратный гессиан вычисляется один раз.

2 Результаты

Исходный код программы представлен в листингах 1– 1.

Листинг 1: Основное задание

```
1 using LinearAlgebra
2 using Plots
3
4 plotly()
5
6 function rosenbrock(x)
7     return (1.0 - x[1])^2 + 100.0 * (x[2] - x[1]^2)^2
8 end
9
10 function rastrigin(x)
11     return 20.0 + x[1]^2 - 10.0*cos(2*pi*x[1]) + x[2]^2 - 10.0*cos(2*pi*
12     x[2])
13 end
14 function schwefel(x)
15     return 418.9829*2 - (x[1]*sin(sqrt(abs(x[1]))) + x[2]*sin(sqrt(abs(x
16     [2]))))
17 end
18 function newton_1d(g, alpha0=0.0; tol=1e-5, max_iter=30, h=1e-5)
19     alpha = alpha0
20     for iter in 1:max_iter
21         df = (g(alpha + h) - g(alpha - h)) / (2 * h)
22         d2f = (g(alpha + h) - 2 * g(alpha) + g(alpha - h)) / (h^2)
23
24         if abs(d2f) < 1e-8 break end
25
26         step = df / d2f
27         alpha = alpha - step
28
29         if abs(step) < tol || abs(df) < tol break end
30     end
31     return alpha
32 end
```

```

33
34 function newton_1d_simplified(g, alpha0=0.0; tol=1e-5, max_iter=30, h=1e
    -5)
35     alpha = alpha0
36
37     #
38
39     d2f = (g(alpha0 + h) - 2 * g(alpha0) + g(alpha0 - h)) / (h^2)
40     if abs(d2f) < 1e-8 return alpha0 end
41
42     for iter in 1:max_iter
43         df = (g(alpha + h) - g(alpha - h)) / (2 * h)
44
45         step = df / d2f
46         alpha = alpha - step
47
48         if abs(step) < tol || abs(df) < tol break end
49     end
50     return alpha
51 end
52 function powell_method(f, x0; line_search_method=newton_1d, max_iter
    =200, eps=1e-4)
53     x = copy(x0)
54     history = [copy(x)]
55     n = length(x)
56
57     D = [zeros(n) for _ in 1:n]
58     for i in 1:n D[i][i] = 1.0 end
59
60     for iter in 1:max_iter
61         P = copy(x)
62
63         #          2:
64
65         for i in 1:n
66             d = D[i]
67             inner_f(alpha) = f(P + alpha * d)
68
69             #
70             best_alpha = line_search_method(inner_f, 0.0)
71
72             P = P + best_alpha * d
73             push!(history, copy(P))
74         end
75     end
76 end

```

```

75         #           3:
76         d_new = P - x
77         if norm(d_new) < eps break end
78
79         #           4:
80
81         inner_f_new(alpha) = f(P + alpha * d_new)
82         best_alpha_new = line_search_method(inner_f_new, 0.0)
83
84         x_new = P + best_alpha_new * d_new
85         push!(history, copy(x_new))
86
87         #
88         for i in 1:(n-1) D[i] = D[i+1] end
89         D[n] = d_new / norm(d_new)
90
91         #
92         D_matrix = hcat(D...)
93         if abs(det(D_matrix)) < 1e-3
94             for i in 1:n
95                 D[i] = zeros(n)
96                 D[i][i] = 1.0
97             end
98         end
99         x = x_new
100     end
101     return x, history
102 end
103
104 function visualize_powell_comparison(f, name, x0, bounds_pad)
105     println("           : $name |           : $x0")
106
107     t1 = time()
108     x_res1, hist1 = powell_method(f, x0, line_search_method=newton_1d)
109     time_std = round(time() - t1, digits=6)
110     iters1 = length(hist1) - 3
111
112     t2 = time()
113     x_res2, hist2 = powell_method(f, x0, line_search_method=
newton_1d_simplified)
114     time_simp = round(time() - t2, digits=6)
115     iters2 = length(hist2) - 3
116
117     println("\ n           :")

```

```

118     println("                                = $iters1 |                                = $(round.(
x_res1, digits=3)) |                                = $time_std                                ")
119     println("                                :")
120     println("                                = $iters2 |                                = $(round.(
x_res2, digits=3)) |                                = $time_simp                                ")
121
122     all_x1 = [p[1] for p in hist1]; all_y1 = [p[2] for p in hist1]
123     all_x2 = [p[1] for p in hist2]; all_y2 = [p[2] for p in hist2]
124
125     min_x = min(minimum(all_x1), minimum(all_x2))
126     max_x = max(maximum(all_x1), maximum(all_x2))
127     min_y = min(minimum(all_y1), minimum(all_y2))
128     max_y = max(maximum(all_y1), maximum(all_y2))
129
130     xr = range(min_x - bounds_pad, max_x + bounds_pad, length=100)
131     yr = range(min_y - bounds_pad, max_y + bounds_pad, length=100)
132     Z = [f([x, y]) for y in yr, x in xr]
133
134     # 2D
135     p1 = contour(xr, yr, Z, levels=50, color=:grays, fill=false, title="$name
(2D)", xlabel="x1", ylabel="x2")
136
137     #
138     plot!(p1, all_x1, all_y1, color=:purple, lw=2, marker=:circle,
markersize=3, label="")
139     scatter!(p1, [x_res1[1]], [x_res1[2]], color=:purple, markersize=7,
marker=:cross, label="")
140
141     #
142     plot!(p1, all_x2, all_y2, color=:orange, lw=2, linestyle=:dash,
marker=:square, markersize=3, label="")
143     scatter!(p1, [x_res2[1]], [x_res2[2]], color=:orange, markersize=7,
marker=:diamond, label="")
144
145     scatter!(p1, [x0[1]], [x0[2]], color=:green, markersize=6, label="")
146
147     # 3D
148     p2 = surface(xr, yr, Z, color=:grays, alpha=0.5, colorbar=false,
title="3D")
149     plot!(p2, all_x1, all_y1, [f(p) for p in hist1], color=:purple, lw
=3, label="")
150     plot!(p2, all_x2, all_y2, [f(p) for p in hist2], color=:orange, lw
=3, linestyle=:dash, label="")
151     scatter!(p2, [x0[1]], [x0[2]], [f(x0)], color=:green, markersize=5,
label="")

```

```

152
153     display(plot(p1, p2, layout=(1,2), size=(1100, 500)))
154 end
155
156 visualize_powell_comparison(rosenbrock, "          ", [-1.2,
1.0], 0.5)
157
158 visualize_powell_comparison(rastrigin, "          ", [0.5, 0.8],
1.0)
159
160 visualize_powell_comparison(schwefel, "          ", [400.0, 400.0],
50.0)

```

Результат запуска представлен на рисунке 1– 3.

ФУНКЦИЯ: Розенброк | Старт: [-1.2, 1.0]

КЛАССИЧЕСКИЙ НЬЮТОН:

Итераций = 12 | Минимум = [1.0, 1.0] | Время = 0.003 сек

УПРОЩЕННЫЙ НЬЮТОН:

Итераций = 3 | Минимум = [-7.52974569420592e11, 1.48216426826453e12] | Время = 0.0 сек

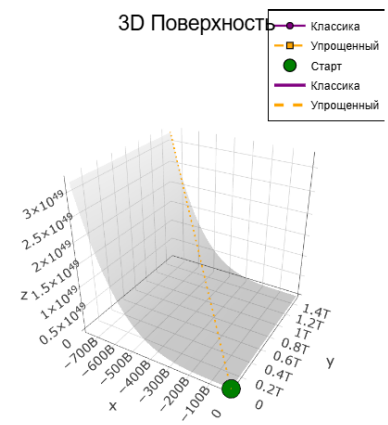
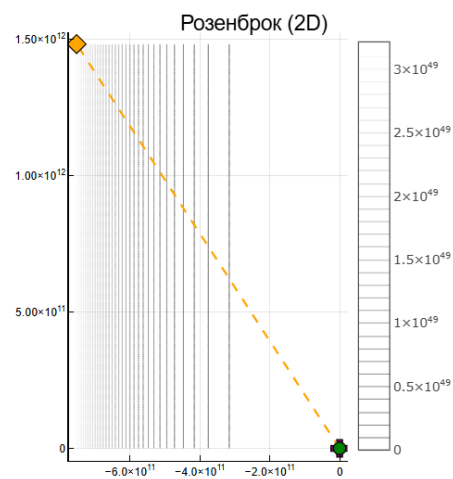


Рис. 1 — Розенброк

ФУНКЦИЯ: Растринг | Старт: [0.5, 0.8]

КЛАССИЧЕСКИЙ НЬЮТОН:

Итераций = 2 | Минимум = [0.503, 1.99] | Время = 0.0 сек

УПРОЩЕННЫЙ НЬЮТОН:

Итераций = 4 | Минимум = [0.503, -3.518] | Время = 0.0 сек

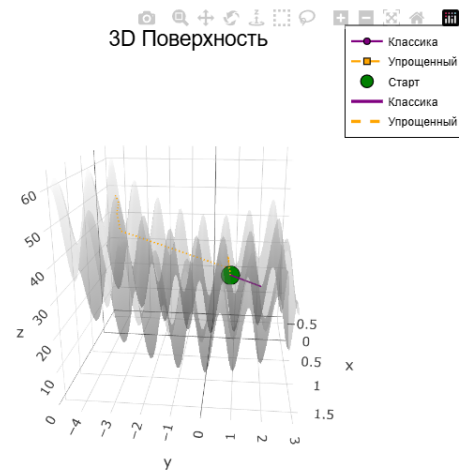
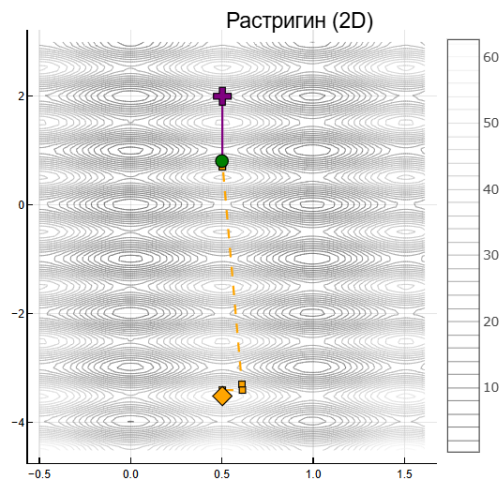


Рис. 2 — Растринг

ФУНКЦИЯ: Швевель | Старт: [400.0, 400.0]

КЛАССИЧЕСКИЙ НЬЮТОН:

Итераций = 2 | Минимум = [420.969, 420.969] | Время = 0.065 сек

УПРОЩЕННЫЙ НЬЮТОН:

Итераций = 2 | Минимум = [420.969, 420.969] | Время = 0.025 сек

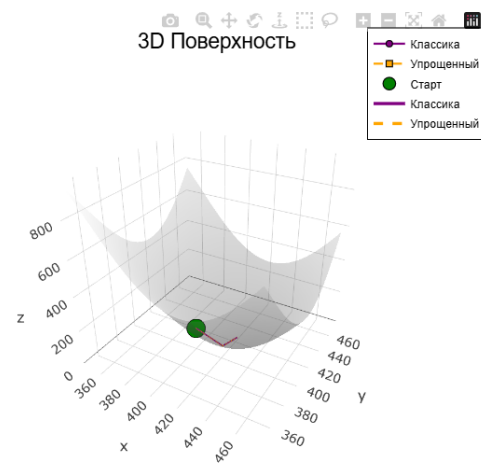
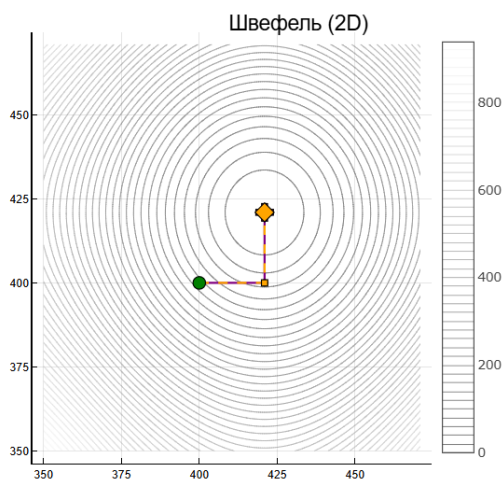


Рис. 3 — Швевель